

CONTENT BROADCASTING SYSTEM

System Architecture & Design Document

Version 1.0 | Prepared for Technical Interview

1. System Overview

The Content Broadcasting System is a web-based platform that allows teachers to upload educational content (images), which goes through an approval process before being broadcast to students' devices in real-time. The system is built with security, reliability, and role-based access as its core design principles.

2. Technology Stack

The following table summarizes the key technologies used and the specific problem each one solves:

Technology	Purpose
Node.js + Express.js	Server runtime and HTTP framework — handles all incoming API requests.
PostgreSQL + Supabase	Relational database — stores users, content metadata, and approval records.
Prisma ORM	Database interface layer — allows writing JavaScript instead of raw SQL queries.
JSON Web Tokens (JWT)	Stateless authentication — lets users stay logged in without server-side session storage.
Bcrypt	Password hashing — ensures stored passwords are never readable, even if the database is compromised.
Zod	Input validation — rejects malformed or missing data before it ever touches business logic.
Helmet.js + CORS	Security headers — protects the API from common web attacks and cross-origin misuse.
Multer	File upload handling — manages incoming image files and saves them to disk safely.
express-rate-limit	Brute-force protection — limits how many requests one IP can make in a time window.

3. Architectural Pattern — Separation of Concerns

The codebase is organized into three distinct layers. Each layer has one responsibility and does not bleed into another. This makes the code easier to test, debug, and scale.

Layer	Responsibility	What it does
Routes	Entry point	Defines URL paths, attaches auth middleware and file upload handlers.
Controllers	Request/response handler	Validates incoming data using Zod, checks user roles, and formats the HTTP response.
Services	Business logic + database	Contains the core logic — what content is live, who can approve, what rotation to show — and talks to the database via Prisma.

4. Authentication & Role-Based Access Control (RBAC)

The system uses a stateless authentication model, meaning the server does not need to store session data. Instead, a signed token (JWT) is issued at login and sent with every request.

How authentication works:

- **Login:** User submits credentials. The password is verified against the stored Bcrypt hash. If valid, a JWT token is returned.
- **Token:** Each token contains the user's ID and role (e.g., Teacher, Principal). It is cryptographically signed so it cannot be tampered with.
- **Request:** Every protected API call must include this token. The server verifies and decodes it before processing anything.

Three-layer access enforcement:

- **Layer 1 — Middleware:** Verifies the token is valid on every incoming request.
- **Layer 2 — Controller:** Checks the user's role. A Teacher attempting a Principal-only action is rejected immediately.
- **Layer 3 — Service:** A final role check inside the business logic itself, as a safety net against any middleware bypass.

Password Security: Passwords are never stored as plain text. Bcrypt hashes them with a configurable cost factor, making brute-force attacks computationally expensive.

Rate Limiting: To protect against automated attacks (brute-force or DDoS), each IP address is limited to 100 requests per 15-minute window.

5. Deterministic Rotation Engine (Live Broadcasting)

This is the most technically interesting component of the system. Rather than storing which content is 'currently playing' in the database (which would require constant writes), the system calculates it mathematically on every request.

Why deterministic?

Any device asking for the current broadcast at the same moment will always get the same answer — without any coordination between requests. This makes the system stateless and highly efficient.

The algorithm (step by step):

- **Filter:** Retrieve only approved content where the current time falls within the item's active time window.
- **Group:** Organize the active items by subject (e.g., Maths, Science).
- **Calculate Cycle:** Add up the total rotation duration of all items in the group.
- **Find Current Position:** Use the current Unix timestamp modulo the total cycle duration to find where we are in the rotation.
- **Select:** Walk through the sorted list and return whichever item's time range the current position falls within.

Performance: The selection runs in $O(n)$ time, where n is the number of active items (typically a very small number). No background jobs or scheduled tasks are needed.

6. Database Design & Data Integrity

- **ORM (Prisma):** Acts as a type-safe bridge between JavaScript code and the PostgreSQL database. Eliminates hand-written SQL and reduces the risk of SQL injection.
- **Singleton Pattern for DB Connection:** Only one database connection pool is ever created, regardless of how many requests come in simultaneously. This prevents connection exhaustion — a critical concern in containerized deployments like Render.
- **Enums for Role & Status:** Values like 'Teacher', 'Principal', 'pending', 'approved', 'rejected' are enforced at the database level, not just in code. This prevents invalid states from ever being stored.
- **User-to-Content Relationship:** Every piece of content is linked to its uploader, providing a full audit trail of who uploaded what and when.

7. File Upload & Static Asset Delivery

- **Storage:** Files are saved to local disk using Multer. Each filename is prefixed with a Unix timestamp to prevent naming collisions.
- **Type Enforcement:** Only JPG, PNG, and GIF files are accepted. The server validates the MIME type on upload, rejecting any other file type.
- **Size Limit:** A hard 10MB cap per file protects the server from memory overload attacks.
- **Delivery:** Files are served via Express's built-in static file handler, which streams the file directly from disk without loading it entirely into memory — efficient and scalable.

8. Content Lifecycle — Approval & Rejection Workflow

Content follows a strict, one-directional state machine to ensure administrative oversight before anything is ever broadcast to students.

State	Who acts	What is recorded
Pending	Teacher uploads content	Upload timestamp, uploader ID
Approved	Principal approves	Approving Principal's ID + timestamp
Rejected	Principal rejects (reason required)	Mandatory rejection reason stored for teacher feedback

Note: Rejection is final. The rejected content is permanently excluded from broadcasting. The teacher must re-upload after addressing the feedback.

9. Edge Case Handling

- **No Active Content:** If no approved content is active at a given moment, the API returns a clean 200 OK response with a descriptive message — not an error code. This prevents student devices from crashing or showing blank screens.
- **Expired Content:** The rotation engine only considers content where the current time falls within the defined start and end window. Expired content is automatically excluded on every request — no manual cleanup needed.
- **Invalid Subject Filters:** If a device subscribes to a subject that doesn't exist or is misspelled, the system gracefully returns an empty result set rather than throwing an error.

10. Student-Facing Broadcast Endpoint

- **Public Access:** The broadcast endpoint requires no authentication, allowing any student device to access it with zero friction.
- **Always Fresh:** Every request triggers a live database query to ensure only currently approved and time-active content is returned.
- **Subject Filtering:** Devices can subscribe to specific content streams via a query parameter (e.g., ?subject=Maths), making it easy to display subject-specific boards.
- **In-Memory Rotation:** The rotation selection is computed in memory at the service layer — it requires no additional database writes, keeping latency low even under concurrent load.